EF Core Persistence

Contents

EF Core Persistence **1**
- Scope . . . 1
- Official requirements . . . 1
- Project policy . . . 1
- Checklist . . . 2
- References . . . 2
- Related . . . 2

EF Core Persistence

Scope

How the `Persistence` project maps the domain, queries it, and guards concurrency. Schema shape and migration process: `../../30-data/database-design-migrations/STANDARD.md`.

Official requirements

Microsoft EF Core guidance is the baseline, in particular the documented behaviour of change tracking, `AsNoTracking`, concurrency tokens and split queries. Provider: `Npgsql.EntityFrameworkCore.PostgreSQL` against Supabase-hosted PostgreSQL (`brain/decisions/ADR-0006-datastore.md`), plus the Npgsql provider's documented behaviour.

Project policy

Mapping

- Configure entities with `IEntityTypeConfiguration<T>` classes, one per entity. No data annotations on domain types — `Domain` must stay persistence-ignorant.
- Explicit table and column names. Do not rely on convention for anything a migration will rename.
- Enums persist as their integer value with an explicit conversion, never as implicit text.
- Every entity has a surrogate primary key plus a `RowVersion`/concurrency token where the entity participates in a concurrency hotspot.

Querying

- Read paths use `AsNoTracking()`. Tracking is for write paths only.
- Project to DTOs in the query (`Select`), never load the entity graph and map afterwards.
- No lazy loading. Explicit `Include` only, and only what the caller needs.
- Every list endpoint is paginated. No unbounded `ToListAsync()`.
- All EF calls are the `Async` overloads and take the request's `CancellationToken`.
- Never interpolate user input into raw SQL. Use parameters or `FromSqlInterpolated`.

Writing

- The transaction boundary is the transaction pipeline behaviour, one per command.
- Concurrency hotspots use **optimistic** concurrency: a concurrency token on the row, `DbUpdateConcurrencyException` handled by reload-and-reapply or by returning `409 Conflict`.
- The one-vote-per-user rule is enforced by a **unique index**, not by a read-then-write check. A read-then-write check is a race condition, and the subject grades exactly this.
- Never increment a counter by reading it into memory first. Recompute from the source rows, or use a database-side update.

Npgsql and Supabase specifics

- Concurrency tokens use the Postgres system column `xmin`, mapped with `.UseXminAsConcurrencyToken()`. No hand-maintained version column.
- Snake-case table and column names are written out in the `IEntityTypeConfiguration<T>`, as the Mapping section above requires. The project pins no naming-convention package: a convention renames silently when a property is renamed, and a silent rename is a migration nobody read. `MusicRoom.Persistence.Tests` walks the built model and fails the build on any name that breaks the `database-design-migrations` conventions.
- `timestamp with time zone` for every instant, always written as UTC.
- The API uses the pooled connection string; `dotnet ef` uses the **direct** one. Enable `No Reset On Close` / disable prepared statements on the pooled connection as Supavisor requires.
- Keep the pool small and connections short-lived. A managed pooler has a hard connection ceiling, and a long-running SignalR process can exhaust it.

Repositories

- Interfaces in `Application`, implementations in `Persistence`.
- A generic base repository handles CRUD and paging; entity-specific repositories add intention-revealing methods. Do not expose `IQueryable` outside `Persistence`.

Checklist

- ☐ `Domain` has no EF Core reference
- ☐ Read queries are `AsNoTracking` and projected
- ☐ Every list endpoint paginated
- ☐ Unique index backs the vote rule; concurrency token on playlist and device rows
- ☐ `DbUpdateConcurrencyException` is handled, not swallowed

References

- EF Core docs — https://learn.microsoft.com/ef/core/
- Concurrency conflicts — https://learn.microsoft.com/ef/core/saving/concurrency
- Efficient querying — https://learn.microsoft.com/ef/core/performance/efficient-querying

Related

`../clean-architecture-cqrs/STANDARD.md` · `../../30-data/database-design-migrations/STANDARD.md`